Viva Preparation — Meridian

Everything an examiner is likely to ask, with the honest answer. Every number here is verifiable from the repository.

Golden rule: never bluff. "We didn't build that, here's why, and here's what happens instead" scores better than a guess that unravels under one follow-up. The design has a defensible reason behind almost every gap — use it.

---

1. One-line answers to have ready

Question	Answer
What is it?	A multi-tenant apartment society management platform
Who uses it?	9 roles across 4 tiers — resident, official, non-resident owner, guard, staff, visitor, super admin
How many epics?	6, covering 24 user stories
Backend?	FastAPI (Python 3.11)
Frontend?	React 18 + TypeScript + Vite
Database?	PostgreSQL on Supabase in production, SQLite locally
How many tables?	18
How many API endpoints?	48 paths, 59 operations
How many tests?	149 backend + 21 ML service
Where's it hosted?	Frontend on Vercel, API + ML on Render, DB on Supabase
What's the ML?	Complaint classifier + payment-risk scorer, scikit-learn
Team size?	4 contributors, 56 commits, 9 branches

---

2. Architecture

Q: Describe your architecture.

Three deployables. A React single-page app, a FastAPI API, and a separate FastAPI ML service. The API is a **modular monolith** — one router per epic — and the ML service is split out because one member owns both models and needs to ship them without touching API releases.

Q: Why not microservices?

At four people, one router per epic gives the same separation of concerns without the operational cost of independent deployables — separate pipelines, versioning between services, distributed transactions. The ML service is the one split we made, and only because it has a genuinely different owner and release cadence.

Q: Why multi-tenant?

One deployment serves many societies, each isolated by `tenant_id`. It's a deliberate expansion beyond the literal brief: it's a materially harder engineering problem — tenant isolation, scope from token claims, a platform admin tier — without much extra feature work, and it matches how real PropTech products are shaped. Retrofitting isolation later is far harder than building with it.

Q: How do you guarantee one society can't see another's data?

Three things:

1. `get_tenant_id` is a FastAPI dependency that reads the tenant **from the JWT claim only** — never from a body or query parameter, so a caller can't ask for another society's data.
2. Every tenant-scoped query filters on it.
3. `tests/test_tenant_isolation.py` attacks every cross-society path with a token belonging to a different society.

Q: Why 404 instead of 403 for cross-tenant reads?

A 403 confirms the record exists. That itself leaks that another society holds that identifier. A 404 says nothing.

---

3. Backend

Q: Walk me through the request lifecycle.

Request → rate-limit middleware → request-id middleware → route dependencies (`get_current_user` → `require_roles` → `get_tenant_id`) → handler → Pydantic response model. Failures go through four exception handlers that produce one uniform error body.

Q: Why FastAPI?

The OpenAPI contract is generated from the route definitions themselves. That's what let the frontend and backend owners work in parallel without a hand-maintained spec drifting. It's also async-capable and Pydantic gives validation for free.

Q: Show me your error handling.

Every failure returns:

```
{ "statusCode": 409, "error": "SLOT_ALREADY_BOOKED",
  "message": "This slot has already been booked", "requestId": "..." }
```

`error` is a stable machine-readable code the frontend branches on — there are **44**, catalogued in the spec as `x-error-catalogue`. `message` is prose we can reword without breaking clients. `requestId` also comes back in an `X-Request-Id` header and appears in the server log, so a user-reported bug traces to its exact log line. The catch-all handler never leaks a stack trace.

Q: What are the routers?

Nine: `auth`, `tenants`, `maintenance`, `complaints`, `bookings`, `visitors`, `announcements`, plus `dashboard` (per-role aggregates) and `ai` (the assistant).

---

4. Security — expect a lot here

Q: How does authentication work?

JWT, HS256. Access token expires in **15 minutes**. Refresh token lasts **7 days** and **rotates** — presenting one issues a new pair and revokes the one presented. So a stolen refresh token is usable at most once, and its reuse is detectable because the stored record is already revoked.

Q: How are passwords stored?

PBKDF2-SHA256, **260,000 iterations**, per-user random salt. The iteration count is embedded in the digest (`pbkdf2_sha256$rounds$salt$hash`), so we can raise the cost later without invalidating existing passwords.

Q: Do you store refresh tokens?

Only their SHA-256 hash. A database leak yields no usable tokens.

Q: How do guest passes work?

A 192-bit random token (`secrets.token_urlsafe(24)`) — not sequential, so passes can't be enumerated. Single-use: the `ACTIVE → USED` flip happens **in the same transaction** that returns the verification result, which is what defeats replay of a photographed QR code. The token is returned only to the resident who created the pass; list views omit it.

Q: How do you prevent payment fraud?

The backend verifies an HMAC-SHA256 over `<order_id>|<payment_id>` with the gateway secret **before** trusting anything in the payload. Without it, a client could mark any invoice paid for free. There's a test that submits a forged signature and asserts the invoice is not settled.

Q: Rate limiting?

Fixed-window, per client IP. **240/min** general, **20/min** on `/auth/*`, **10/min** on `/sos`. Auth is where credential stuffing lands; `/sos` fans out to every guard over push and SMS, so a loop is both a cost and an alert-fatigue problem.

If asked why fixed and not sliding: a fixed window admits a burst across a boundary — up to twice the limit. We accepted that because it costs one `INCR` against Redis where a sliding-window log costs a sorted-set write plus a trim, and the threat is a sustained loop rather than a precisely timed burst.

If asked about `X-Forwarded-For`*:* only trusted behind a proxy we've explicitly marked trusted. A client can set that header themselves, so trusting it unconditionally would let anyone reset their own counter.

Q: Any known security gaps?

Complaint attachments were never implemented, so there's no file-upload attack surface. Everything else on our list is a feature gap rather than a vulnerability. The demo passwords are public by design and would be rotated for real use.

---

5. Database

Q: How many tables and why that design?

18. Every tenant-scoped table carries `tenant_id`, and every composite index leads with it, because every query filters on it first.

Q: Why VARCHAR + CHECK instead of native Postgres enums?

Values stay readable in the database, and altering a native enum is a migration cost we didn't want every time a status was added.

Q: How do you prevent double-booking? *(very likely — it's in Milestone 1)*

Two defences, and they're not interchangeable:

1. A **per-slot lock** — Redis when configured, in-process otherwise — so the loser gets a clean `409` instead of a database error.
2. A **partial unique index** on `slot_id WHERE status = 'CONFIRMED'`, which makes a second confirmed booking *impossible* at the database level.

The index is the real guarantee; the lock is an optimisation. It's **partial** so cancelling frees the slot. Tested with **eight genuinely simultaneous threads** — exactly one `201`, seven `409`s.

Q: Migrations?

Alembic. But there are two paths to the schema — `create_all` locally, Alembic on Postgres — and they can drift. So `tests/test_migrations.py` builds the schema **both ways and compares them structurally**: columns, foreign keys, indexes, CHECK constraints. A model change without a migration fails in CI, not on deploy.

Q: Why SQLite locally?

The project has to run for an evaluator with no database server. `DATABASE_URL` switches to Postgres, and CI applies the migrations against a real Postgres container.

---

6. Machine learning — be careful and honest here

Q: What ML did you build?

Two models in a separate service:

- **Complaint classifier** — TF-IDF (word uni- and bigrams) into logistic regression, one model each for category and urgency.
- **Payment risk** — gradient-boosted trees over five features.

Q: Why TF-IDF and logistic regression, not a neural network?

Right-sized for the data. A few thousand short texts with six classes — a linear model on TF-IDF trains in seconds, is interpretable, and serves in milliseconds. Bigrams matter specifically: *"not urgent"* and *"no water"* both invert what their individual words suggest.

Q: What accuracy did you get?

Category macro-F1 **1.0**, urgency **0.99**, payment risk ROC-AUC **0.67**.

⚠️ **Immediately add the caveat — don't wait to be caught:**

> That 1.0 is **not** a generalisation estimate. Training and test data come from the same phrase templates, so it largely measures the templates. We tested against phrasings written nowhere in the training set and it got 4 of 6 — and importantly, returned confidence 0.30 and 0.39 on the two it missed, below the backend's 0.55 threshold, so the fallback caught them. Our test asserts *never confidently wrong* rather than *never wrong*, because a wrong-but-confident answer is the only genuinely damaging outcome — it routes a complaint to the wrong trade without flagging it.

Q: 0.67 ROC-AUC is poor, isn't it?

The Bayes-optimal for that generative process is **0.68** — the model reaches 98% of what's achievable. Outcomes are sampled from a probability, so most of the remaining gap is irreducible noise. The trainer records both numbers so the headline isn't read out of context.

Q: Where did your training data come from?

107 hand-written seed phrases expanded with openers, locations and urgency cues into ~2,650 examples. The risk model is trained on a simulated history. **Both are bootstraps** — retraining on real complaint text and invoice history is what would make the metrics evidence rather than a demonstration.

Q: What if the ML service is down?

Complaint submission **never** fails. It falls back to a deterministic keyword classifier and flags the complaint `needs_manual_triage`. Same on timeout, bad response, or confidence below threshold. That's designed behaviour, not a failure mode — and it's the pattern used for every external dependency.

---

7. Testing

Q: How do you test?

`pytest` with FastAPI's `TestClient`. **Each test gets its own throwaway SQLite database**, so tests share no state and run in any order — no PostgreSQL, no Redis, no running server. That's what makes the suite runnable from a fresh clone.

Q: Which tests are you proudest of?

Three that test *properties* rather than endpoints:

Suite	What it proves
`test_tenant_isolation.py`	Every cross-society path, attacked with a foreign token
`test_bookings.py`	8 real threads on one slot → exactly one winner
`test_migrations.py`	Migrations and models build identical schemas

Q: Did testing find real bugs?

Yes — three worth naming.

TC-11: an expired guest pass was still listed as `ACTIVE`. The validity window was checked on the verify path but not the list path. We documented it with root cause, held it under a `strict=True` xfail so it couldn't be silently fixed, scheduled it, fixed it, and the case is now a passing assertion.

Two in the Redis lock, found by testing against a real Redis rather than a fake:

- Release used `EVAL` unconditionally; some managed Redis disables scripting, so the lock sat until its TTL and stalled every booking for that slot.
- A *contended* lock and an *unreachable* Redis were handled identically, so a request that lost the race barged in anyway — two threads measurably inside the critical section.

Q: Frontend tests?

None — this is our largest genuine gap. `tsc --noEmit` is a real gate but it's a type gate, not a behaviour gate. Vitest and React Testing Library, then Playwright for the golden paths, is the top item in our future work.

---

8. Frontend

Q: Why two state libraries?

They solve different problems. **TanStack Query** holds server state — caching, refetching, invalidation. **Zustand** holds session state — the signed-in user and tokens — which needs none of that and must be readable *outside* React by the API client. Mixing them would mean either caching the session or hand-rolling cache invalidation.

Q: How does the frontend know the API's shape?

It's generated. `backend/openapi.yaml` → `openapi-typescript` → `schema.d.ts`. CI regenerates and fails if it differs from what's committed, so the app can't typecheck against a contract the API no longer serves.

Q: How does auth work in the browser?

The API client attaches the bearer token, unwraps errors into a typed `ApiError`, and refreshes an expired access token before replaying the request. The refresh is **single-flight** — several queries commonly 401 in the same tick, and because the backend revokes a refresh token as it rotates it, letting each refresh independently would log the user out.

---

9. Deployment

Q: Where is it deployed?

Frontend on **Vercel**, API and ML service on **Render**, database on **Supabase Postgres**, optional Redis on **Upstash**.

Q: How does the frontend reach the API?

Vercel rewrites `/api/*` to Render, so the browser stays **same-origin** in production exactly as it does behind the Vite dev proxy locally. CORS is never exercised, and no API URL is compiled into the bundle — the same build works against any backend.

Q: How does the database get its schema?

`alembic upgrade head` runs in the start command, before the server. If it fails the service never serves — deliberately. An API answering against a half-built schema is worse than one that's plainly down.

Q: Why is the first request slow?

Render's free tier sleeps after ~15 minutes idle; waking takes 30–50 seconds. Known and documented, not a performance defect.

---

10. Process and teamwork

Q: How did you track work?

Jira — 5 epics, 24 work items, 6 sprints from June to August. **13 commits carry their Jira id** in the subject line, so board items trace to the code.

Q: How did you do code review?

Branch → PR → CI → peer review → merge. Three PRs, all merged, one explicitly approved, authored by two different people. CI is the automated first pass: lint, tests, typecheck and contract-drift run before a human looks at the diff.

Q: Did user feedback change anything?

Yes, and one case is worth telling. An official asked to skip the `IN_PROGRESS` state when staff fix something on the spot. **We declined** — that intermediate state is what makes the audit trail meaningful — and instead built a one-tap action that issues *both* transitions. They get one click; the history stays complete.

Another became the `?needs_triage=true` filter. A third was the TC-11 defect.

---

11. Weaknesses — rehearse these

Being asked "what's wrong with it?" is an opportunity, not a trap.

Gap	The honest answer
No frontend tests	Largest real gap. `tsc` is a type gate, not a behaviour gate. Vitest → RTL → Playwright is next.
Notifications don't send	Routing is implemented and logged at the dispatch boundary; no provider is wired. The in-app row is always written, so nothing is lost — only its external copy.
No WebSockets	Clients poll. Every REST endpoint behind the intended events exists and is tested; only the transport is missing.
ML on synthetic data	Both models are bootstraps. We report metrics with that caveat rather than at face value.
No file uploads	`attachment_urls` exists but there's no object store behind it.
No PWA	Scoped out.
Anaemic models	The class diagram shows entity methods; we kept domain logic in the router/service layer, where the transaction boundary is. Deliberate, not an oversight.

---

12. If you're asked something you don't know

1. Say so plainly.
2. Say where the answer lives — "that's in `app/core/security.py`" or "Part G of the report covers that".
3. Offer the nearest thing you *do* know.

Never invent a number. Every figure in this document is checkable in about thirty seconds, and so is a wrong one.

---

13. Numbers worth memorising

```
9 roles · 6 epics · 24 user stories
48 API paths · 59 operations · 62 schemas · 44 error codes
18 tables · 13 indexes
149 backend tests · 21 ML tests
15-minute access token · 7-day rotating refresh · 260,000 PBKDF2 rounds
192-bit guest pass token · single-use
8-thread concurrency test · exactly 1 winner
Rate limits: 240 general / 20 auth / 10 SOS per minute
ML: category F1 1.0 (templated) · urgency 0.99 · risk AUC 0.67 vs 0.68 Bayes
56 commits · 4 contributors · 9 branches · 3 PRs
```